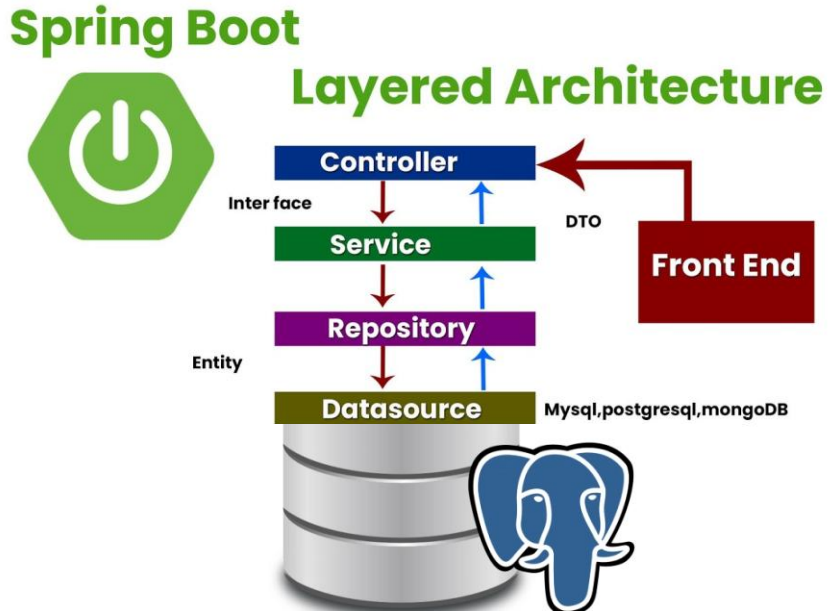
Høgskolen på Vestlandet



DAT108
Programmering og webapplikasjoner

F20 - Spring DI og Spring Data JPA

Lars-Petter Helland, 23.10.2026

I dag

- Oppsummering av hva vi har sett på så langt

- Spring IoC-container
  - Dependency Injection (DI), Beans, Scopes, @Service
- Spring Data JPA 
- Lagdeling i Spring-applikasjoner
  - @Controller, @Service, Repository

F15 Web Oversikt Intro

Generelle begreper:

Internett, IP-adresse, domene, DNS, protokoll, webben, HTTP, URL, port, sti, request-parametre, (web)klient, (web)server

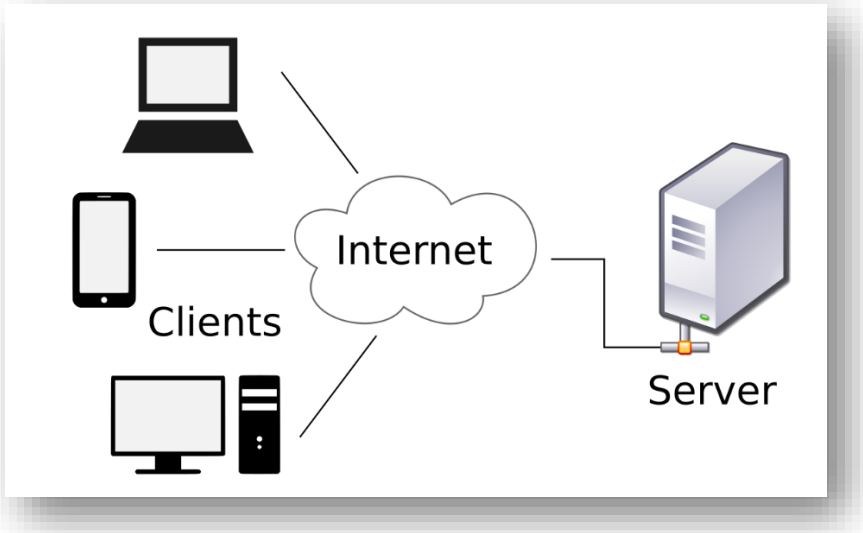
HTTP:

Request (fra klient) – **Response** (tilbake fra server)

Hvordan vi lager/rendrer HTML-sider på serveren. Dette kalles Server-Side-Rendering (SSR), og gjøres via et oppsett som heter ..

Model-View-Controller (MVC):

- **Controller** tar imot request og lager klar en model
- **Model** inneholder relevante data for requesten
- **View** bruker dataene i model til å generere en response



F16 Webapps med Spring Boot og Spring MVC

- Hva er Spring, Spring Boot, Spring MVC
- Prosjektoppsett med maven og pom.xml
- Spring MVC `@Controller`-klasse

```
@GetMapping("/sti")
public String m(Model model, String pnavn) { ...
```

- Controller kan legge til attributter i model

```
model.addAttribute("hilsen", "Hallo");
```

- Et Thymeleaf View har tilgang til model-attributter slik:

```
<p th:text="${hilsen}"></p>
```

- Et Thymeleaf View kan også iterere over samlinger og trekke ut egenskaper fra objekter slik:

```
<p th:forEach="p : ${personliste}"
        th:text="|Hallo ${p.navn}|"></p>
```





F17 Requester og Form-input

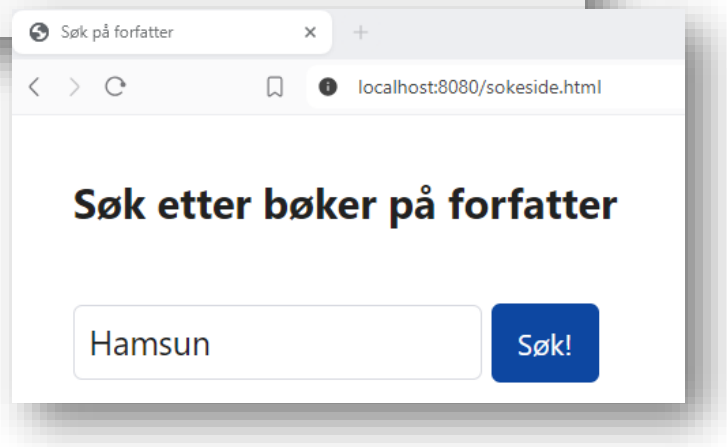
GET request (m/parametre i URL)

Brukes for å hente informasjon

POST request (m/parametre i request-body)

Brukes for å sende/lagre informasjon

```
<form action="noenboker" method="get">
    <input type="text" name="forfatter">
    <input type="submit" value="Søk!">
</form>
```



HTML **<form action=".. " method="..">**

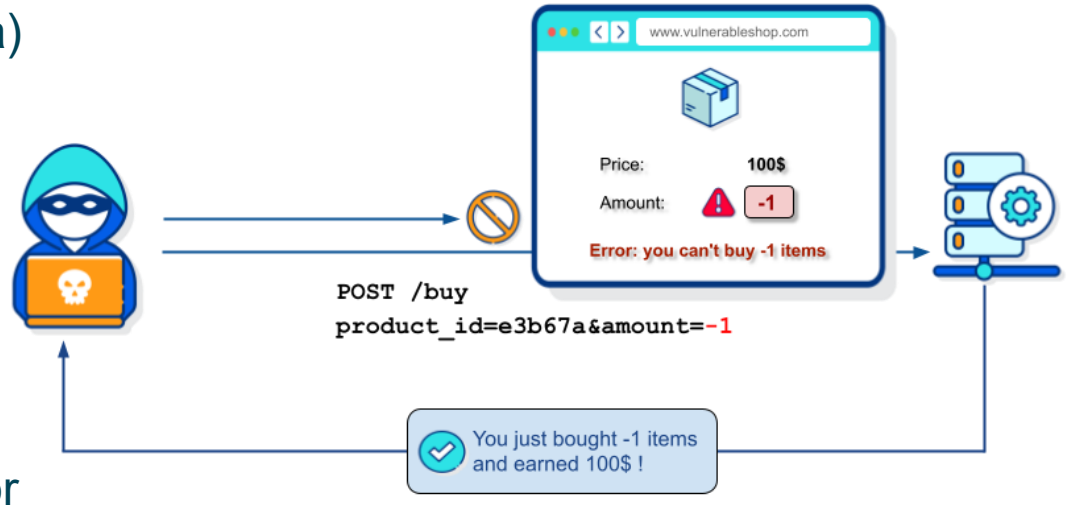
... **<input type=".." name="..">**:

text, password, number, **submit**, image, radio, **checkbox**, date, email, url, **hidden**, <textarea>, <select>

Alle request-parametre blir sendt over som **tekst**. I controlleren må vi sjekke at de nødvendige parametre er med, og at de er gyldige!

F18 Validering av brukerinput

- Validering på klienten (i nettleseren, med JavaScript) handler om brukervennlighet.
- **Validering på serveren** (ved mottatt request, med Java) handler om robusthet og sikkerhet.
- **Programmatisk validering**: Vi lager en hjelpeklasse med metode(r) som sjekker om ulike typer input er gyldig i hht. regler vi har bestemt.
- **Data Binding og Deklarativ validering**: Vi binder request-parametrene (ved navnelikhet) til egenskaper i et objekt, og vi bruker annoteringer for å angi reglene for hva som er gyldig. Vi bruker @Valid @ModelAttribute og BindingResult for automatisk validering.
- **Regulære uttrykk** er et nyttig verktøy i validering.



F19.1 PRG Sesjoner Innlogging - PRG

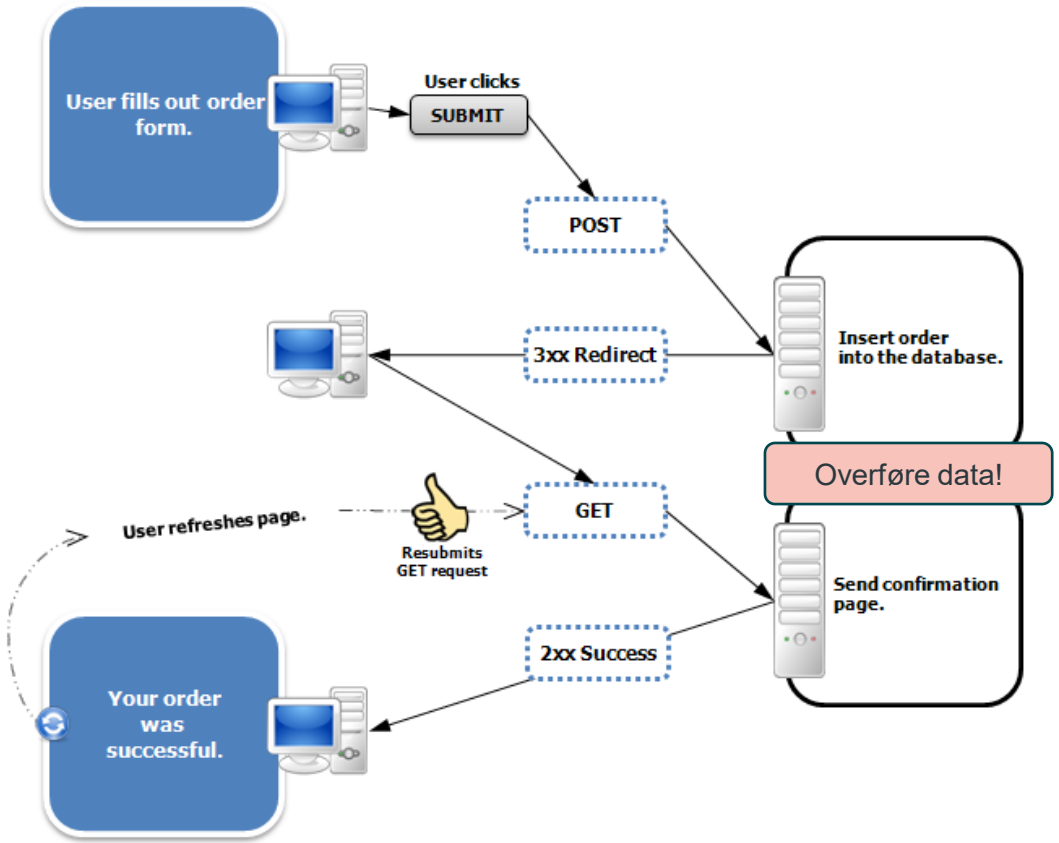
- PRG: Ved en POST-request bør man alltid redirecte i stedet for å gi en respons (via et view). Dette for å unngå dobbeltposting.
- I controlleren gjøres dette ved

```
return "redirect:kvittering";
```

der **kvittering er en sti** (ikke navnet på et view) !

- Data som skal «overleve» fra POST-request til GET-request etter redirect må tas vare på på én av disse måtene:

```
//Legge på som request-parameter
ra.addAttribute("vare", vare);
//Legge til modell i redirect-request
ra.addFlashAttribute("vare", vare);
```



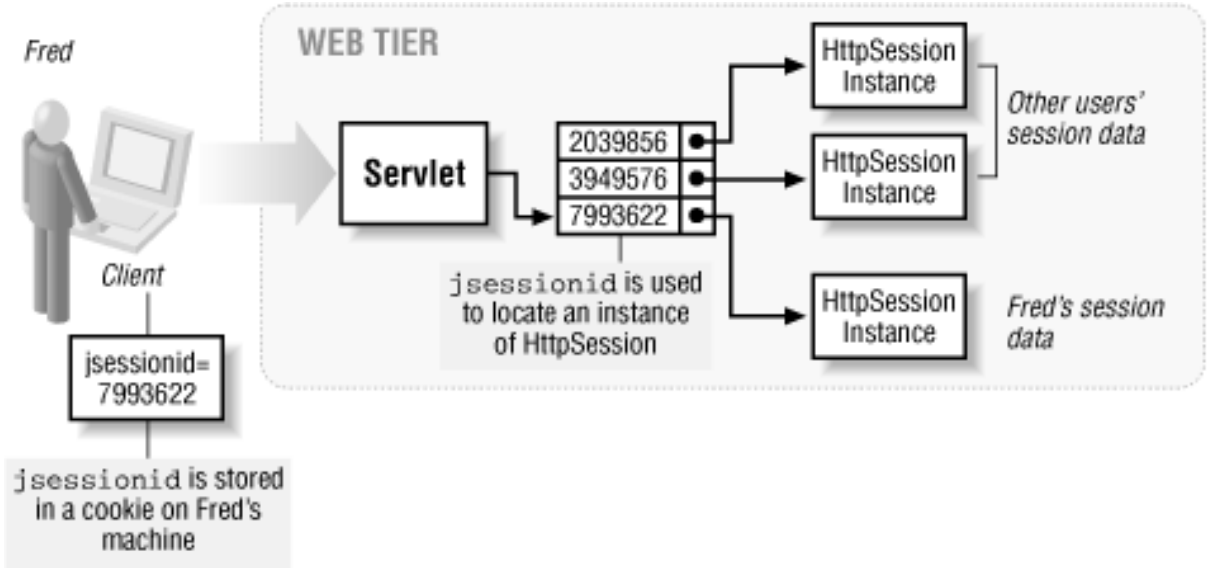
F19.2 PRG Sesjoner Innlogging - HttpSession

- Vi kan lagre data på serveren (i et HttpSession-objekt) knyttet til en gitt session-id.
- session-id sendes til brukeren som en cookie, og vil deretter sendes tilbake til serveren i påfølgende requests.
- HttpSession-APIet inneholder bl.a.:

```
sesjon.setAttribute(navn, objekt);
T data = (T) sesjon.getAttribute(navn);
sesjon.setMaxInactiveInterval(sekunder)
```

- Sesjonsattributter hentes frem med Thymeleaf slik:

```
<p th:text="${session.navn}"> ..
```



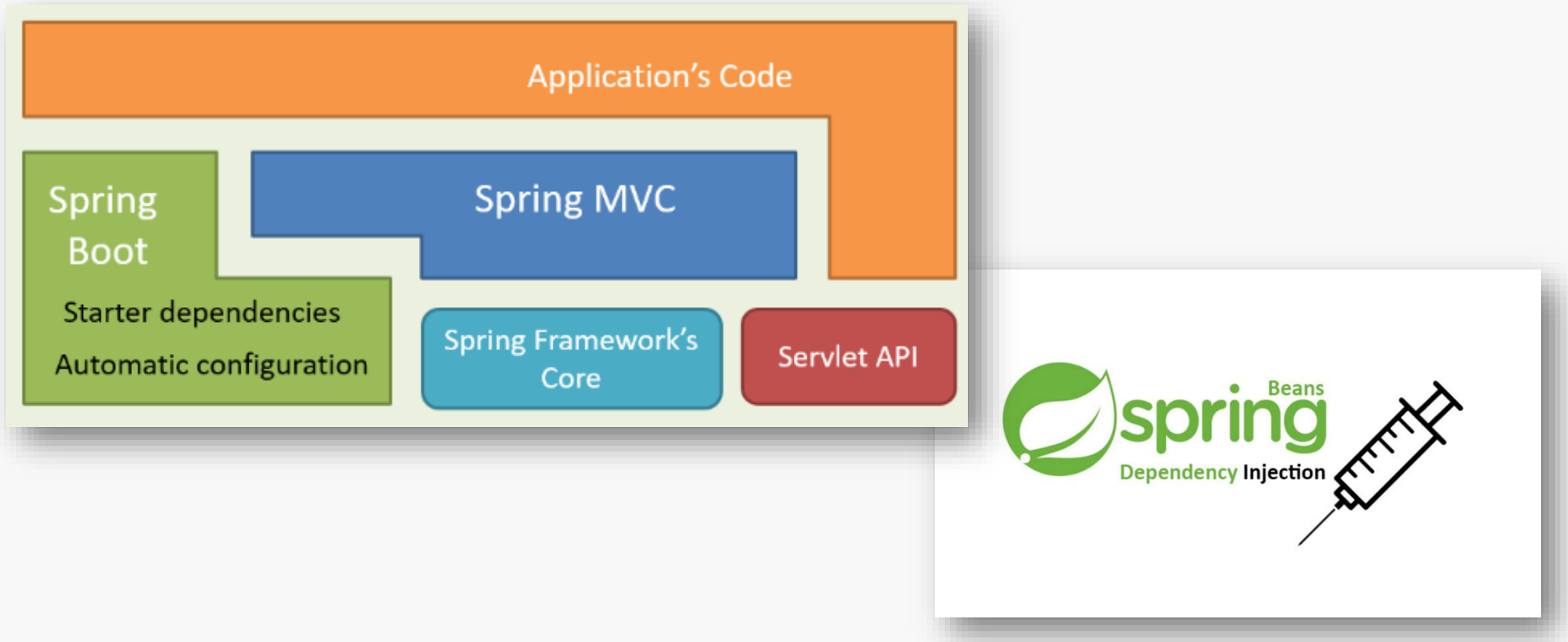
Hva gjenstår av semesteret

Uke	Forelesning mandag	Javahjelp (B) tors	Forelesning fredag	Lab - Litt ulike tider B, H og F
43 **19.10-...**		JH10 (kun Bergen)	**F20** Web6 **Spring DI, Data JPA, lagdeling**	**Oblig4** - **Frist søndag**
44 **26.10-...**	NY! **F21 Hente data fra REST-API** (Lars-Petter kan sikkert ta denne)	JH11 (kun Bergen)	NY! **F22 REST1 tilsv. DAT152 (Tosin) sin WS1** (Lars-Petter)	Oblig5 - REST m/ Spring MVC
45 **02.11-...**	NY! **F23 REST2 tilsv. DAT152 (Tosin) sin WS2** (Lars-Petter)	JH12 (kun Bergen)	NY! **F24 Testing av Spring webapplikasjoner #1**	**Oblig5** - **Frist tirsdag over helgen**
46 **09.11-...**	F25 Passord og autentisering. Også litt ift. REST-API (Lars-Petter)		F26 Testing av Spring webapplikasjoner #2	
47 **16.11-...**	**F27** Gml. eksamen **H25**. Alle lærerne. Streames til Førde og Haugesund			
50 (E) **07.12-...**	**DAT108** **Q&A mandag 07.12 !?**	**DAT108** **Eksamen onsdag 09.12**		

Nå



Spring Core, Spring MVC, Spring Boot, Spring DI



Hva er Spring?

Spring er et **rammeverk for Java-utvikling**. Ved å bruke et rammeverk trenger vi kun å skrive kode for deler av programmet. Rammeverket tar seg av resten. Koblingen mellom Spring-rammeverket og koden gjøres via det som kalles **Dependency Injection** *(kommer straks tilbake til)*. Spring består av en rekke moduler.

Spring MVC er en modul på toppen av **Java Servlet API-et** *(standard Java-API for webutvikling)* og legger til rette for å utvikle web-applikasjoner basert på **Model View Controller**-prinsippet.

Spring Boot gjør konfigurasjon og kjøring av Spring-applikasjoner mye enklere ved å tilby mange hjelpeklasser og default-innstillinger, og videre enkel konfigurasjon via properties- eller yaml-filer.

Dependency Injection (DI)

(ide til forklaring ca. fra Hands-on Spring Boot Primer, Kap.5)

Hva er en **Dependency** (avhengighet)?

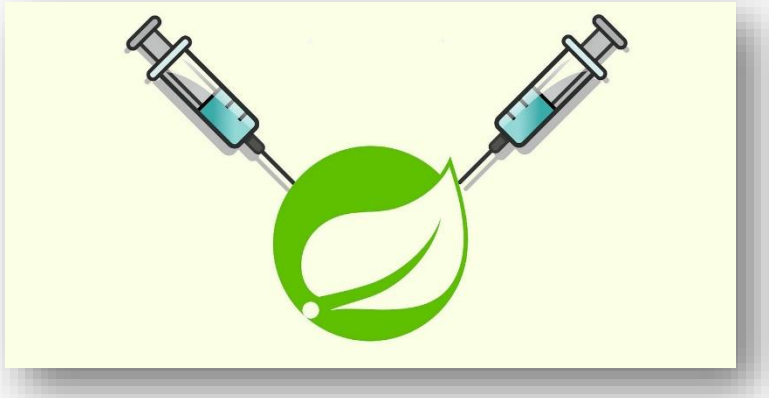
Avhengighet betyr at en klasse (A) refererer til og bruker en instans av en annen klasse eller interface (B), f.eks.

```
public class A { private B b; ...}
```

Hva er **Injection** (injeksjon)?

Injeksjon beskriver det at klassen (A) knyttes til en instans av den andre klassen, en subklasse av denne eller en implementasjon av interfacet (hvis B er et interface) som **gis** utenfra. Dette kan f.eks. gjøres i konstruktøren til A slik

```
public class A { private B b;
    public A(B b){
        this.b = b;
    }
}
```



Interfaces og løs kobling mellom objekter

- Ved å **bruke interfaces og ikke konkrete klasser** oppnår man det man kaller **løs kobling (loose coupling)**. Dette er noe vi ønsker fordi det gjør programmet vårt fleksibelt.
- Eksempel med løs kobling (List er et interface) og dependency injection (avhengigheten til den konkrete klassen blir injisert via konstruktøren. Klassen A trenger ikke å vite om dette er en ArrayList, en LinkedList eller noe helt annet):

```
public class A {
    private List<String> liste;

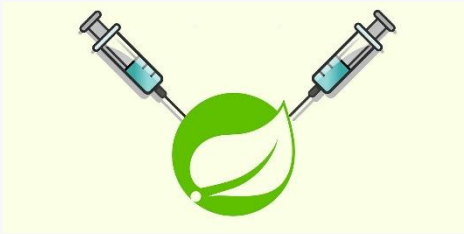
    public A(List<String> liste) {
        this.liste = liste;
    }
}
```

Spring DI

- Dependency Injection i Spring styres til dels av annoteringer.
- Når Spring starter opp, scannes alle annoterte klasser (kalt komponenter) slik at objekter av disse automatisk kan injiseres der vi har oppgitt at vi ønsker det.
- Følgende annoteringer omfattes av Spring sin komponent-scanning:
  - **@Service**
  - **@Controller**, @RestController
  - @Component, @Configuration, @Bean
  - @ControllerAdvice, @ManagedBean @Named @Mapper
  - **...**
- I tillegg vil Spring generere kompontenter for aktuelle:
  - **Repository, CrudRepository, JpaRepository**

Konstruktør-injeksjon

- Komponenter kan automatisk injiseres på flere ulike måter. Den som er anbefalt er **konstruktør-injeksjon**.
- Eksempel:



```
@Controller
public class A {
    private final MyBean mybean;
    public A(MyBean mybean) {
        this.mybean = mybean;
    }
}
```

- Under kjøring vil da objektvariabelen mybean i A automatisk bli satt til å referere et objekt av typen MyBean (eller subklasse eller implementasjon av MyBean) som Spring har gitt oss.

Objekters Scopes (hvor og når det er tilgjengelig. levetid)

- Når vi injiserer et objekt via konstruktøren, betyr det at controlleren deretter alltid refererer **samme objekt?**, eller betyr dette at vi kan få et **nytt objekt for hver request?**, eller kan det også bety **noe midt i mellom?**
- Svaret er litt teknisk, men for de interesserte, sjekk ut «Dynamic Proxy».
- Rekkevidde/levetid/**Scope** til objekter i Spring styres via annoteringer, og er en sentral del av Spring DI. Vi skal ikke gå nærmere inn på det, men ..
- Scopes i Spring:
  - @Scope("**prototype**") betyr at vi får et nytt objekt hver gang.
  - @Scope("**singleton**") betyr at det kun er ett objekt av denne typen. Får det samme hver gang.
  - @**Request**Scope (kun for web) betyr at objektet kun gjelder i denne requesten.
  - @**Session**Scope (kun for web) betyr at objektet (kun) gjelder i denne sesjonen.
  - @**Application**Scope (kun for web) betyr at objektet gjelder i hele applikasjonen.



f20a-validator-som-service

- Vi har hittil kun sett på komponent-typen **@Controller**.
- En annen vanlig komponent-type i Spring er **@Service**.
- Både kontrollere og servicer er Spring-komponenter med **Singleton**-scope.
- Vi skal se at **hjelpeklasser i Spring bør lages som Service**, i stedet for som en klasse med static-metoder.
- Fordeler ved å gjøre det slik vil vi se flere eksempler på senere.





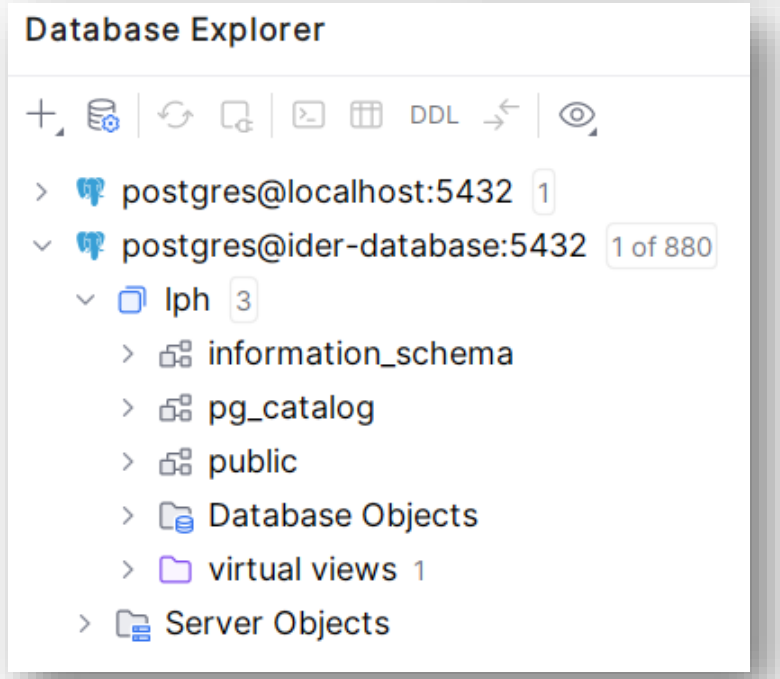
Spring Data JPA

Friske opp hjernebarken – DB, DataGrip, ..

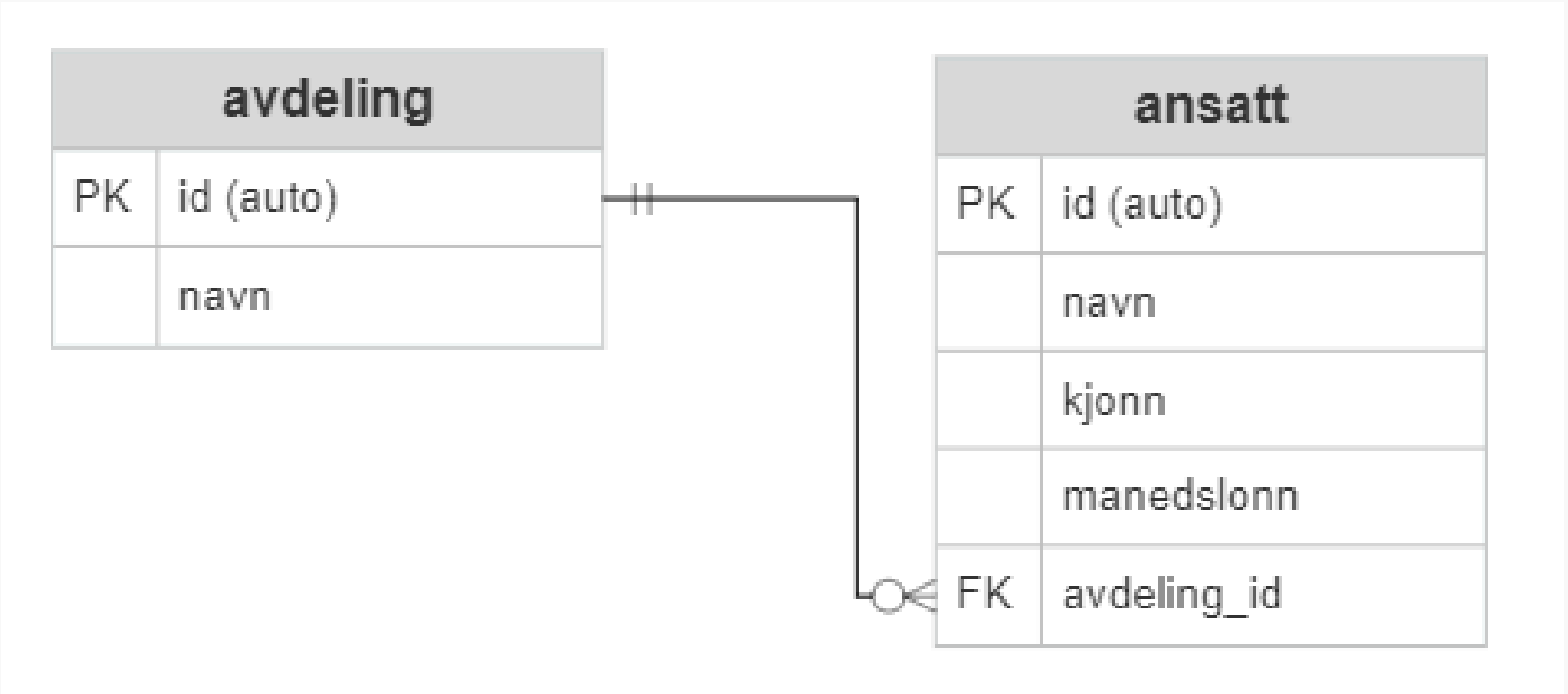
Dere har alle en en database på serveren vår:

```
Server: ider-database.westeurope.cloudapp.azure.com
Port: 5432
Database: h123456 (ditt studentnr)
Bruker: h123456 (ditt studentnr)
Passord: pass (eller det du har endret til)
```

Dere skal bruke denne nå i DAT108.



Vi tar utgangspunkt i et enkelt eksempel



Tabelldefinisjoner

```sql
CREATE TABLE avdeling
(
    id SERIAL PRIMARY KEY,
    navn VARCHAR(30) NOT NULL
);

CREATE TABLE ansatt
(
    id SERIAL PRIMARY KEY,
    navn VARCHAR(30) NOT NULL,
    kjonn CHAR CHECK (kjonn='M' OR kjonn='K'),
    manedslonn INTEGER NOT NULL,
    avdeling_id INTEGER NOT NULL,
    FOREIGN KEY (avdeling_id) REFERENCES avdeling(id)
);
```

Entity-klasser

```java
@Entity
@Table(schema = "ansatt_avdeling")
public class Avdeling {
   @Id @GeneratedValue(strategy= GenerationType.IDENTITY)
   private Integer id ;
   private String navn;
```

```java
@Entity
@Table(schema = "ansatt_avdeling")
public class Ansatt {
   @Id @GeneratedValue(strategy= GenerationType.IDENTITY)
   private Integer id ;
   private String navn;
   private String kjonn;
   private int manedslonn;

   @ManyToOne @JoinColumn(name = "avdeling_id")
   private Avdeling avdeling;
```

Vi begynner enkelt med å kun se på avdeling

```
CREATE TABLE avdeling (
    id SERIAL PRIMARY KEY,
    navn VARCHAR(30) NOT NULL
);
```

CRUD-operasjonene på denne kan f.eks. se slik ut:

- C: public Avdeling opprettAvdeling(String navn)
- R: public Avdeling finnAvdelingMedId(int id)
- R: public Avdeling finnAvdelingMedNavn(String navn)
- R: public List<Avdeling> finnAlleAvdelinger()
- U: public void endreNavnPaaAvdeling(String navn)
- D: public void slettAvdeling(int id)

Hvordan gjorde vi i DAT107?

```
public class Databasehjelper {
    private EntityManagerFactory emf = Persistence.createEntityManagerFactory( ... );
    public Avdeling opprettAvdeling(String navn) {
        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();
        Avdeling avdeling = new Avdeling(navn);
        try {
            tx.begin();
            avdeling = em.persist(avdeling);
            tx.commit();
        } catch (Throwable e) {
            e.printStackTrace();
            tx.rollback();
        } finally {
            em.close();
        }
        return avdeling;
    }
```

Transaksjons-håndtering!

Kontroll av EM sin livssyklus!

Unntaks-håndtering!

Jeg lovde at det kom til å bli enklere i DAT108

Når vi kjører på en web-server vil denne håndtere mesteparten av de "tjenestene" vi trenger for å få gjort jobben, f.eks.:

- EntityManager sin livssyklus.
- Unntakshåndtering.
- Transaksjonshåndtering.

Hvordan vil samme eksempel se ut da?

Samme eksempel på en web-tjener

```
@Repository @Transactional
public class Databasehjelper {
    @Autowired private EntityManager em;
    public Avdeling opprettAvdeling(String navn) {
        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();
        Avdeling avdeling = new Avdeling(navn);
        try {
            tx.begin();
            avdeling = em.persist(avdeling);
            tx.commit();
        } catch (Throwable e) {
            e.printStackTrace();
            tx.rollback();
        } finally {
            em.close();
        }
        return avdeling;
    }
```

Eller enda mer kompakt

```
@Repository @Transactional
public class Databasehjelper {

    @Autowired private EntityManager em;

    public Avdeling opprettAvdeling(String navn) {
        return em.persist(new Avdeling(navn));
    }
    ...
}
```

Men Spring Data JPA har enda mer i ermet for oss

Siden CRUD-operasjoner på entiteter er så vanlig å gjøre har Spring definert et par interfaces vi kan bruke:

`CrudRepository<T,ID>` (basic crud-operasjoner)

- `save(T entitet)` *NB! Brukes også til oppdatering*
- `findById(ID id)`
- `findAll()`
- `deleteById(ID id)`
- ... med en del flere ...

`JpaRepository<T,ID>` (extends CrudRepository, med enda flere nyttige operasjoner, f.eks. paging og sortering)

I stedet for at vi skriver vårt Repository for hånd, kan vi bruke et slikt.

... extends JpaRepository (eller CrudRepository)

```
public interface AvdelingRepo extends JpaRepository<Avdeling, Integer> {}
```

Det er alt! :)

Da vil Spring implementere et slikt JpaRepository for oss som vi kan bruke f.eks. slik:

```
@Controller
public class AvdelingController {

    private final AvdelingRepo avdelingRepo;

    //Dere husker: Dependency Injection i konstruktøren
    public AvdelingController(AvdelingRepo avdelingRepo) {
        this.avdelingRepo = avdelingRepo;
    }
    ...
```

pom.xml og application.properties

pom.xml:

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
</dependency>
```

application.properties:

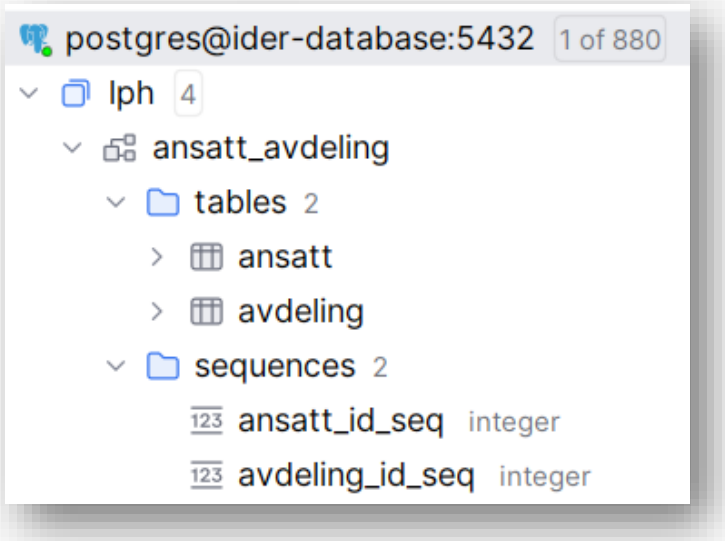
```
spring.datasource.url=jdbc:postgresql://ider-database.westeurope.cloudapp.azure.com:5432/h123456
spring.datasource.username=h123456
spring.datasource.password=pass
```



f20b-avdeling-crudrepo – Kun avdeling

- Ser først på et eksempel kun med `Avdeling`
- Vi starter med å kjøre sql-skript, f.eks. i DataGrip, for å opprette databasen
- Deretter kan vi kjøre web-applikasjonen som er koblet mot denne databasen ...
- og teste ut i nettleseren:

```
localhost:8080/avdelinger
localhost:8080/avdpaaid?id=1
localhost:8080/avdpaanavn?navn=Salg
localhost:8080/nyavd?navn=Drift
```






NEXT

Nå se på Avdeling og Ansatt i kombinasjon

```java
@Entity
public class Avdeling {
@Id @GeneratedValue(strategy= GenerationType.IDENTITY)
private Integer id ;
private String navn;
```

```java
@Entity
public class Ansatt {
@Id @GeneratedValue(strategy= GenerationType.IDENTITY)
private Integer id ;
private String navn;
private String kjonn;
private int manedslonn;

@ManyToOne @JoinColumn(name = "avdeling_id")
private Avdeling avdeling;
```

Et utvalg av operasjoner i tillegg til ren CRUD

Generelt vil vi ha behov for mer sammensatte operasjoner mot databasen enn helt enkle CRUD-operasjoner. Hvilke dette er, avhenger av applikasjonen og hvilken funksjonalitet vi ønsker. F.eks. kan det være:

- `public int snittlonnIAvdeling(int avdId)`
- `public double kvinneandelIAvdeling(int avdId)`
- `public void flyttAnsattTilAvdeling(int ansattId, int avdId)`
- `public Ansatt nyansattIAvdeling(String navn, String kjonn, int manedslonn, int avdId)`
- `public void lonnsoppgjor(Function<Ansatt, Integer> nyLonnBeregning)`
- `...`

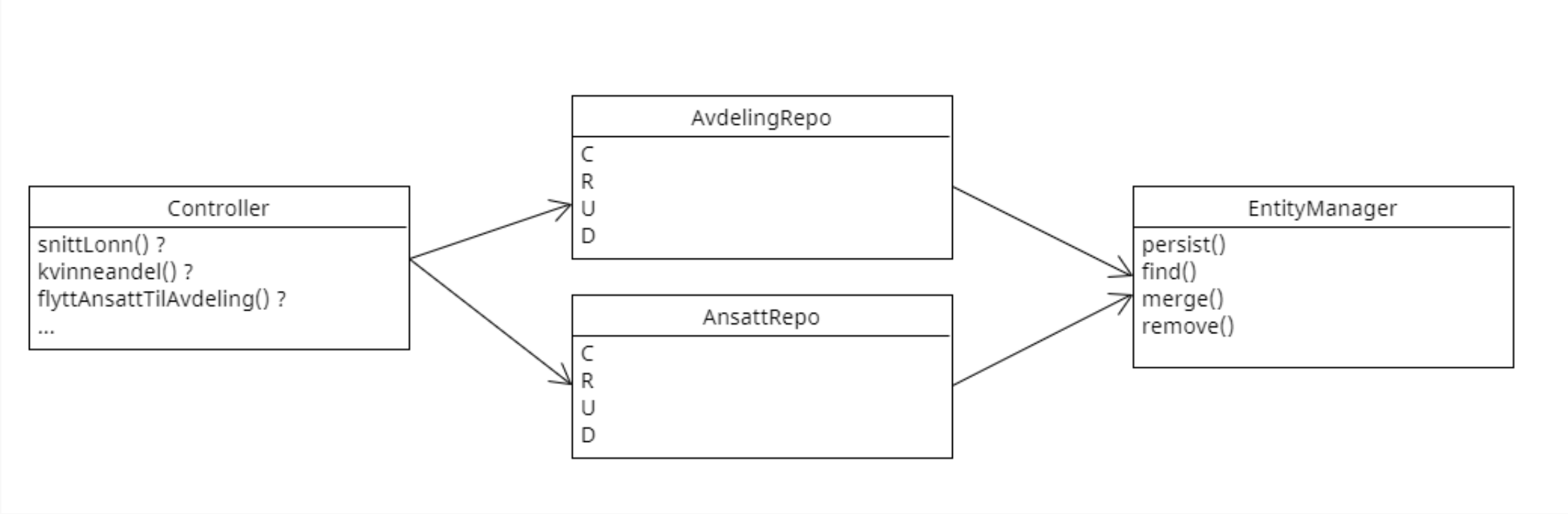
Hvordan kan/bør applikasjonen struktureres når vi har slik **forretningslogikk** i tillegg til Controller og Repository?

Lagdeling



Struktur og lagdeling - Alt.1: Controller ↔ selvlagede Repos

Med egendefinerte klasser for data-aksess. Hvor legger vi metodene flyttAnsattTilAvdeling() osv...? I Controller? I AvdelingRepo? I AnsattRepo?

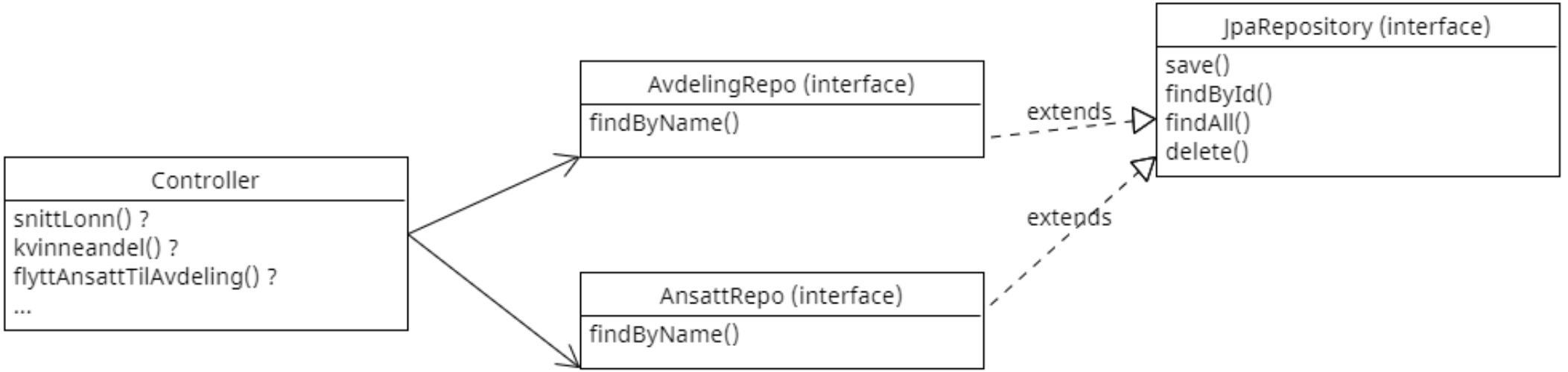


Struktur og lagdeling - Alt.2 : Controller ↔ Spring JpaRepos

Men vi ønsker jo kanskje benytte oss av Crud-/JpaRepository interfacene.

Da er det jo ikke naturlig/mulig å legge snittLonn() osv. i Repo-ene.

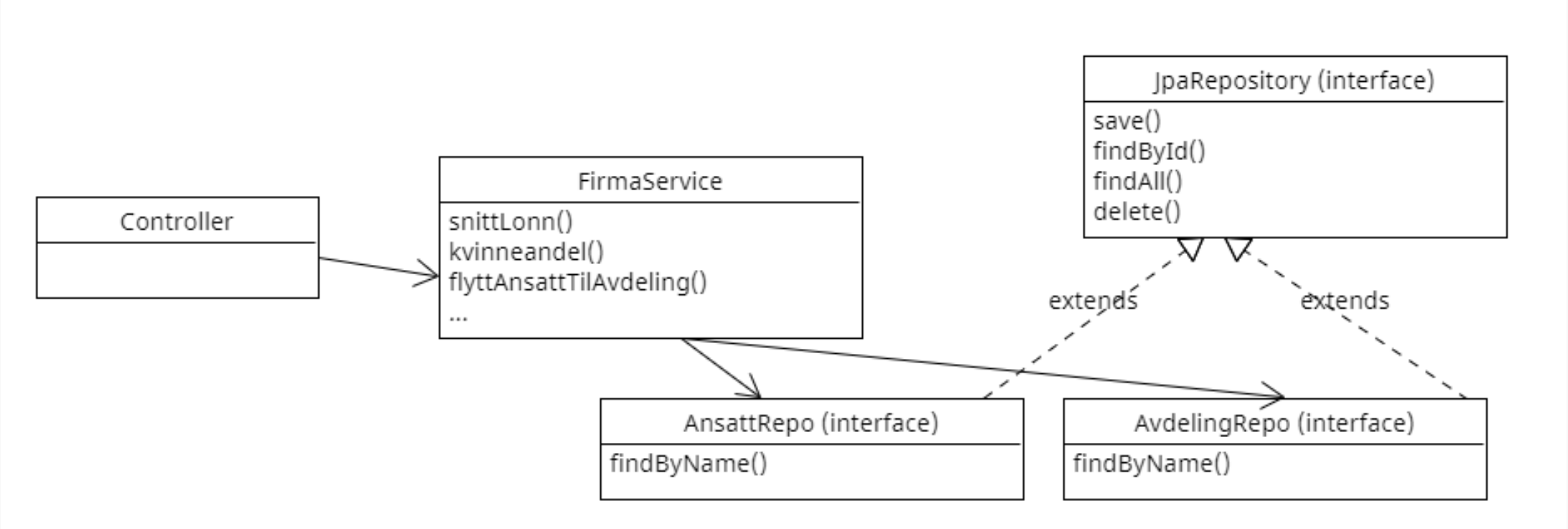
Men de bør jo ikke ligge i Controller heller. Så hva gjør vi?



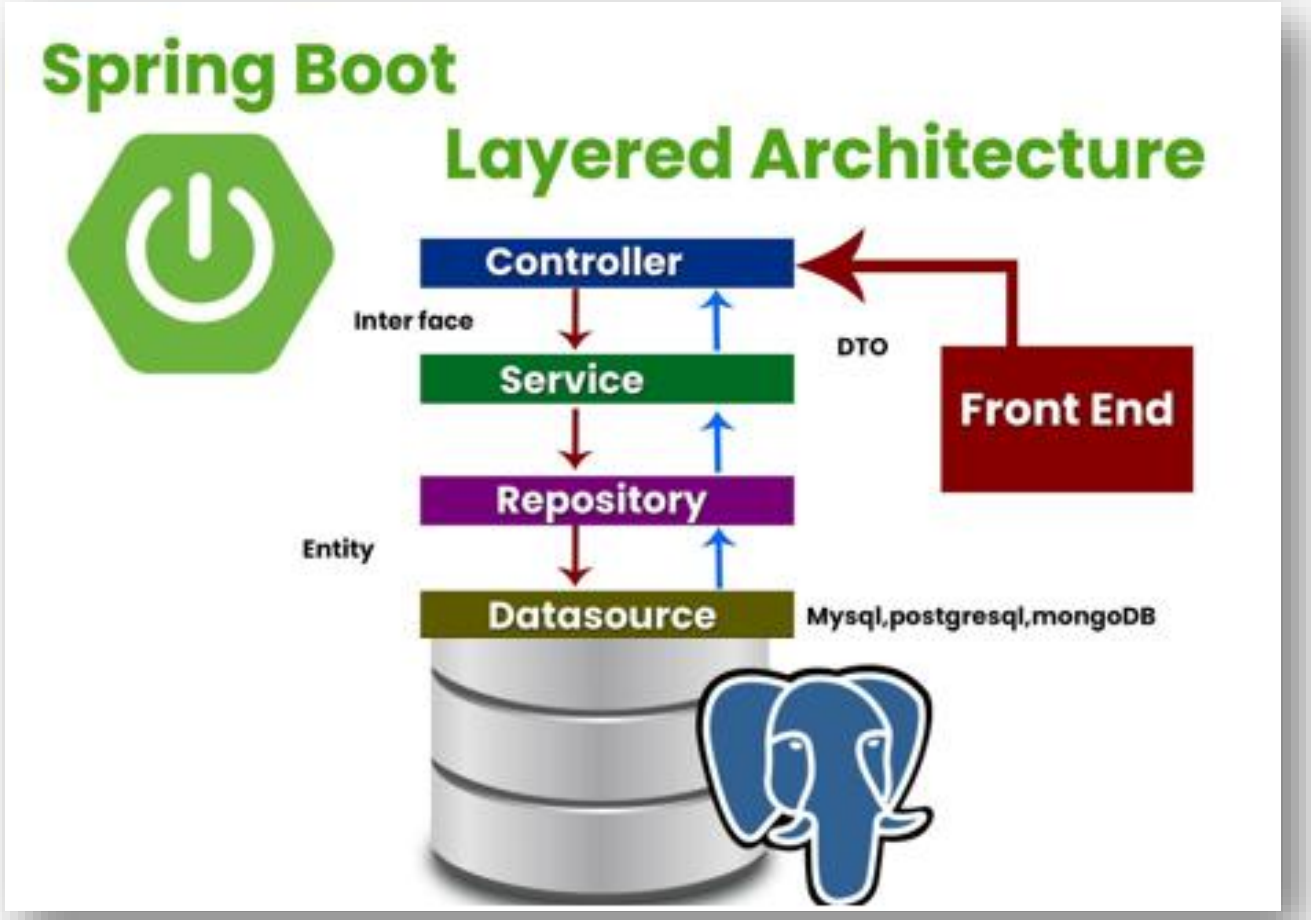
Struktur og lagdeling - Alt.3: **Controller ↔ Service ↔ JpaRepos**

Spring har definert en anbefalt arkitektur der vi har en klar **lagdeling** mellom

Presentation-, **Business**- og **Persistence**-layer. En @Controller (presentation) bør ikke snakke direkte med et @Repo (persistence) , men gå via en @Service (business), slik:



Standard lagdeling i Spring-applikasjoner



f20c-ansatt-avdeling-service

- Vise



Videre

- ... **Oblig4** inn søndag
- Neste uke begynner vi på modul 5 om API-er og REST
- God helg !

